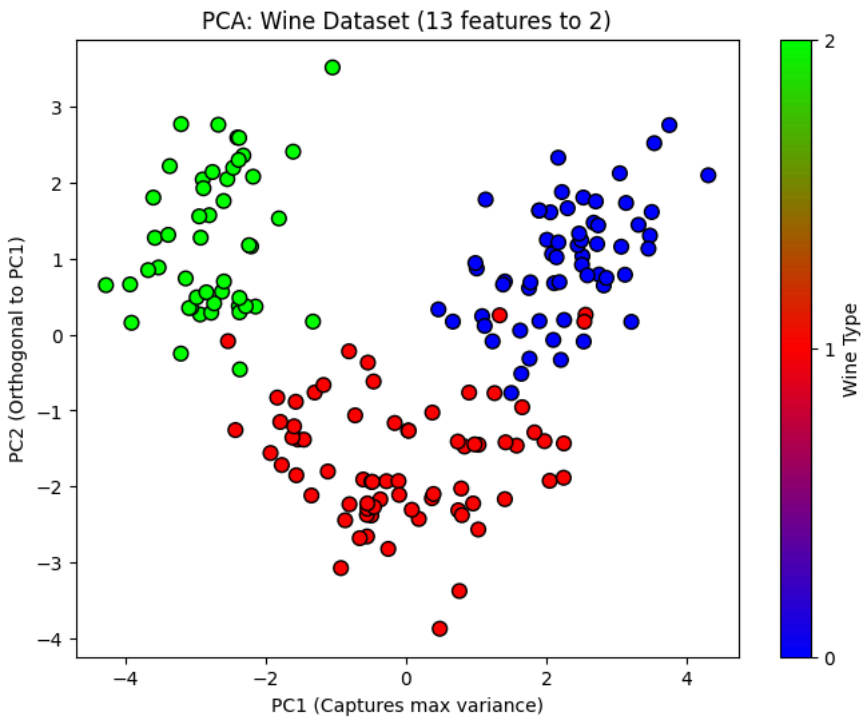
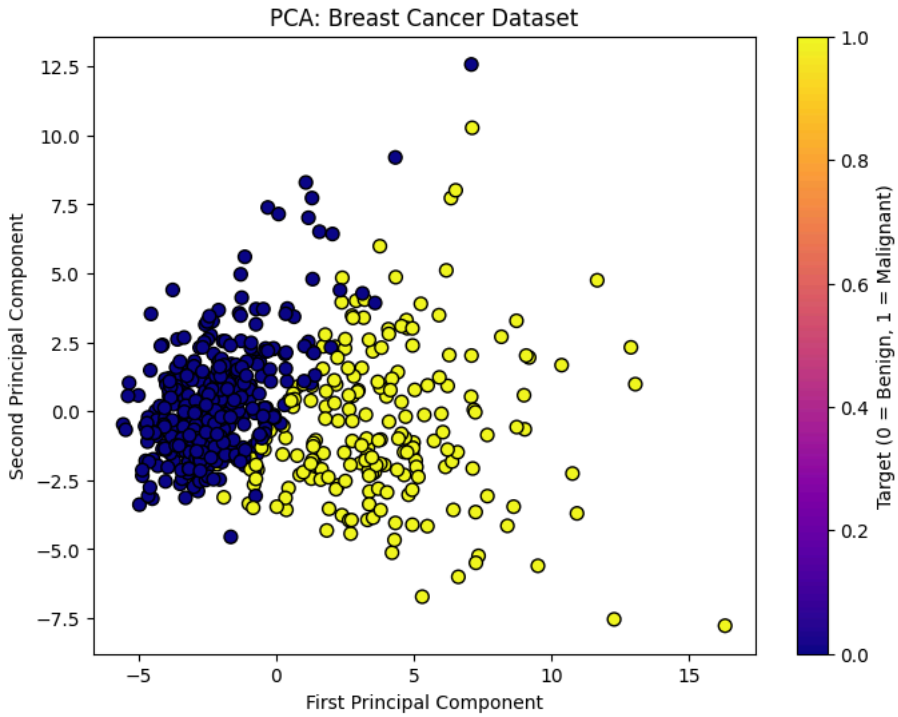
```
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
# Load data
wine = load_wine()
X, y = wine.data, wine.target
# PCA requires standardized data (Mean=0, Var=1)
X_scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='brg', edgecolor='k',
s=60)
plt.title('PCA: Wine Dataset (13 features to 2)')
plt.xlabel('PC1 (Captures max variance)')
plt.ylabel('PC2 (Orthogonal to PC1)')
plt.colorbar(ticks=[0, 1, 2], label='Wine Type')
plt.show()
```




```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Load data
data = pd.read_csv('/content/breast_cancer_data.csv')

# Drop useless column(s)
data = data.drop(columns=['id', 'Unnamed: 32'], errors='ignore')

# Target column
y = data['diagnosis'].map({'M': 1, 'B': 0})

# Feature columns
X = data.drop('diagnosis', axis=1)

# Check if any NaN still exists
print(X.isnull().sum().sum())

# If any NaN remains, remove rows
X = X.dropna()
y = y.loc[X.index]

# Standardize
X_scaled = StandardScaler().fit_transform(X)

# PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
# Plot
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='plasma', edgecolor='k', s=50)
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('PCA: Breast Cancer Dataset')
plt.colorbar(label='Target (0 = Benign, 1 = Malignant)')
plt.show()

# Explained variance
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

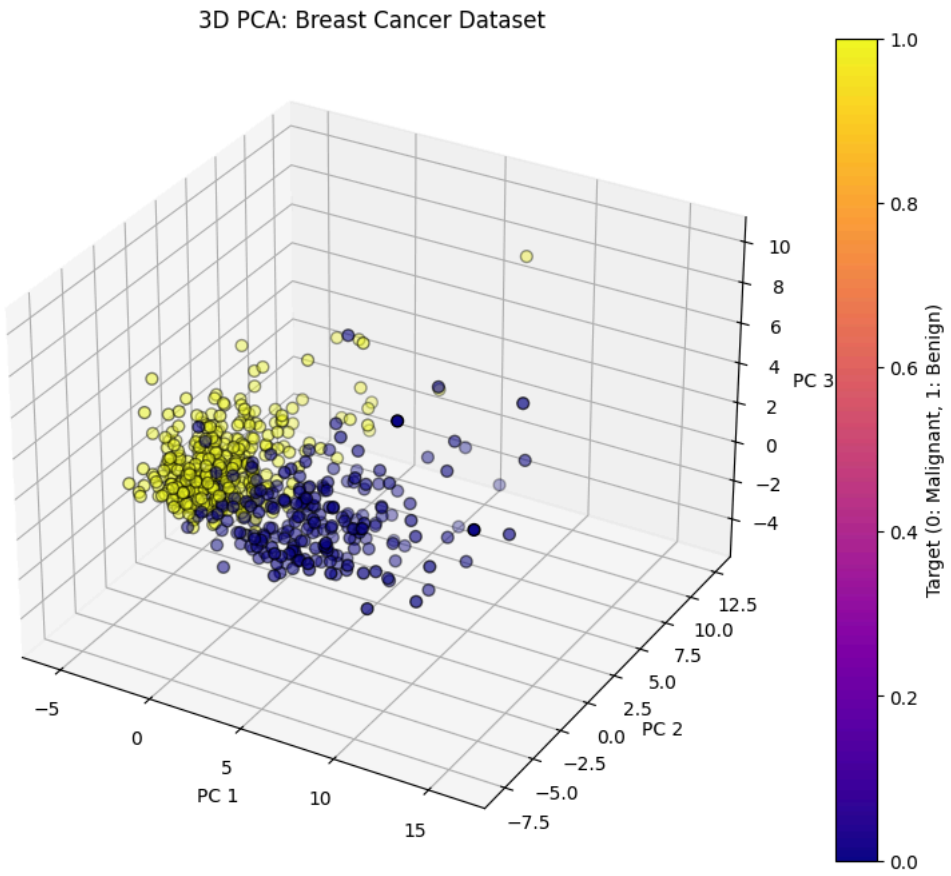
```
0
```




```
Explained Variance Ratio: [0.44272026 0.18971182]
```

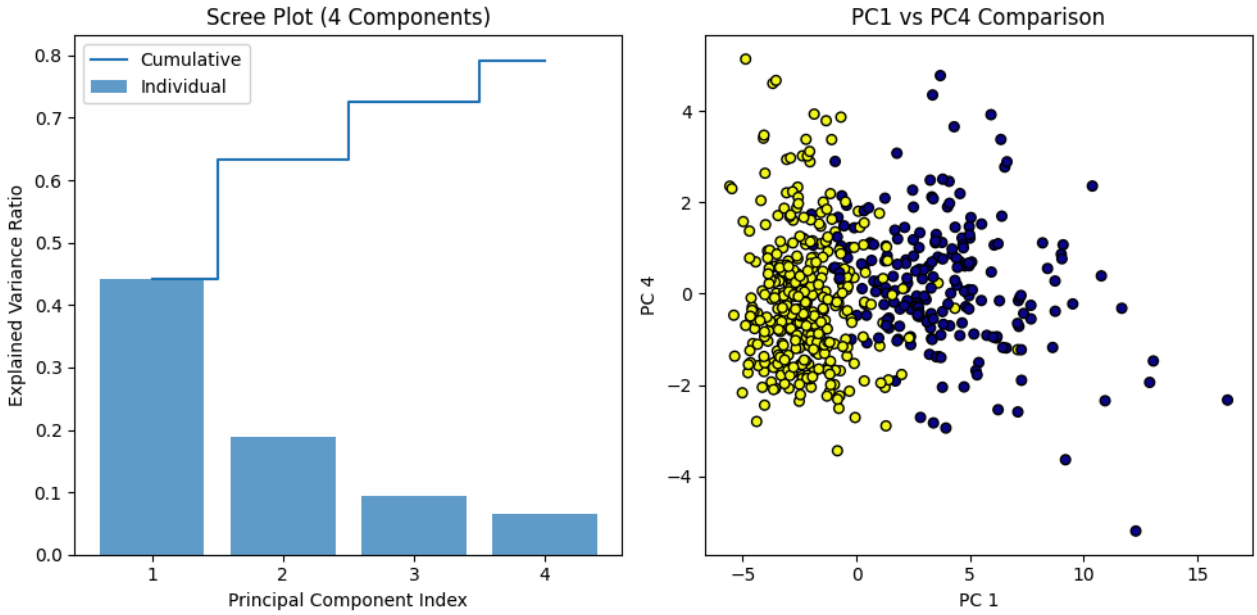
```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.datasets import load_breast_cancer
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Load and scale data
data = load_breast_cancer()
X, y = data.data, data.target

# Standardize the features
X_scaled = StandardScaler().fit_transform(X)

# PCA with 3 components
pca_3 = PCA(n_components=3)
X_pca_3 = pca_3.fit_transform(X_scaled)

# 3D Scatter Plot
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    X_pca_3[:, 0],
    X_pca_3[:, 1],
    X_pca_3[:, 2],
    c=y,
    cmap='plasma',
    edgecolor='k',
    s=40
)

ax.set_xlabel('PC 1')
ax.set_ylabel('PC 2')
ax.set_zlabel('PC 3')
ax.set_title('3D PCA: Breast Cancer Dataset')

# Color bar
```

```
plt.colorbar(scatter, label='Target (0: Malignant, 1: Benign)')
plt.show()

# Print variance info
print("Explained Variance Ratio (3 components):", pca_3.explained_variance_ratio_)
print(f"Total Variance Captured: {sum(pca_3.explained_variance_ratio_) * 100:.2f}%")
```



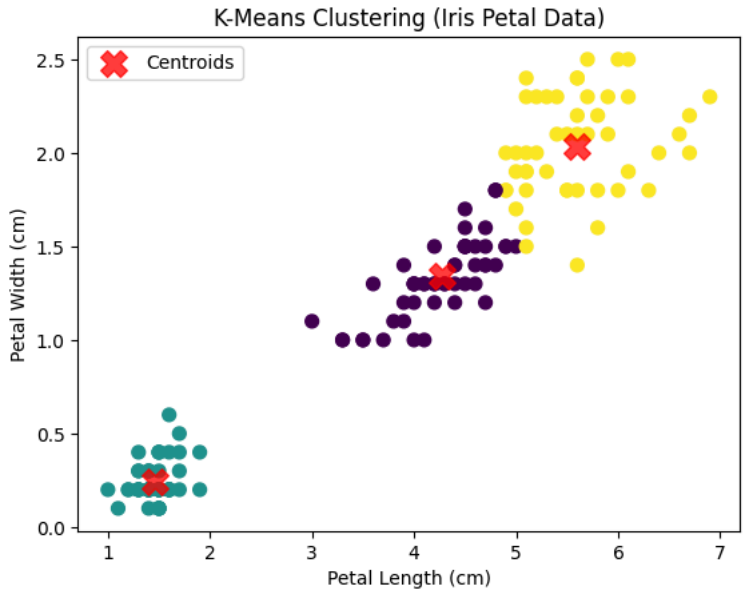

```
Explained Variance Ratio (3 components): [0.44272026 0.18971182 0.09393163]
Total Variance Captured: 72.64%
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

# PCA with 4 components
pca_4 = PCA(n_components=4)
X_pca_4 = pca_4.fit_transform(X_scaled)

# 1. Scree Plot
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.bar(range(1, 5), pca_4.explained_variance_ratio_, alpha=0.7, label='Individual')
plt.step(range(1, 5), np.cumsum(pca_4.explained_variance_ratio_), where='mid', label='Cumulative')
plt.ylabel('Explained Variance Ratio')
plt.xlabel('Principal Component Index')
plt.title('Scree Plot (4 Components)')
plt.xticks(range(1, 5))
plt.legend()

# 2. PC1 vs PC4 Scatter
plt.subplot(1, 2, 2)
plt.scatter(X_pca_4[:, 0], X_pca_4[:, 3], c=y, cmap='plasma', edgecolor='k', s=30)
plt.xlabel('PC 1')
plt.ylabel('PC 4')
plt.title('PC1 vs PC4 Comparison')

plt.tight_layout()
plt.show()

# Variance info
print("Explained Variance Ratio (4 components):", pca_4.explained_variance_ratio_)
print(f"Total Cumulative Variance: {np.sum(pca_4.explained_variance_ratio_) * 100:.2f}%")
```

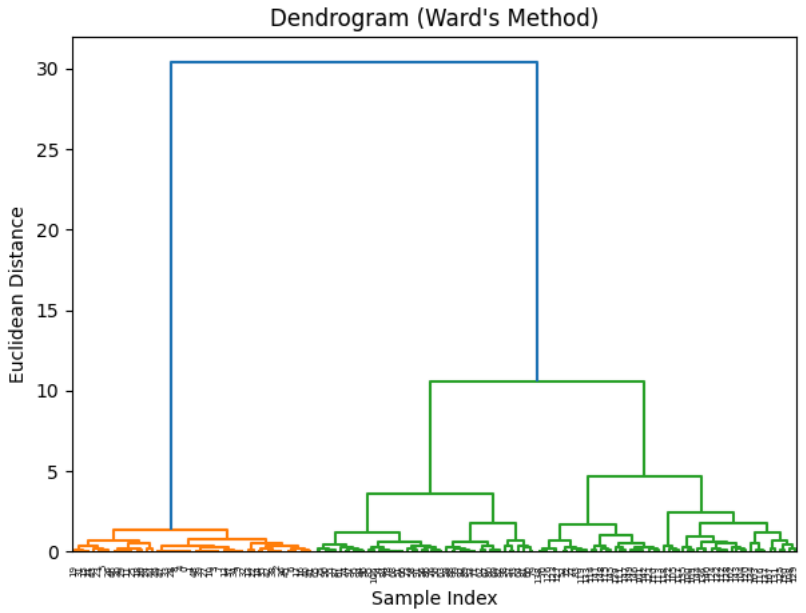


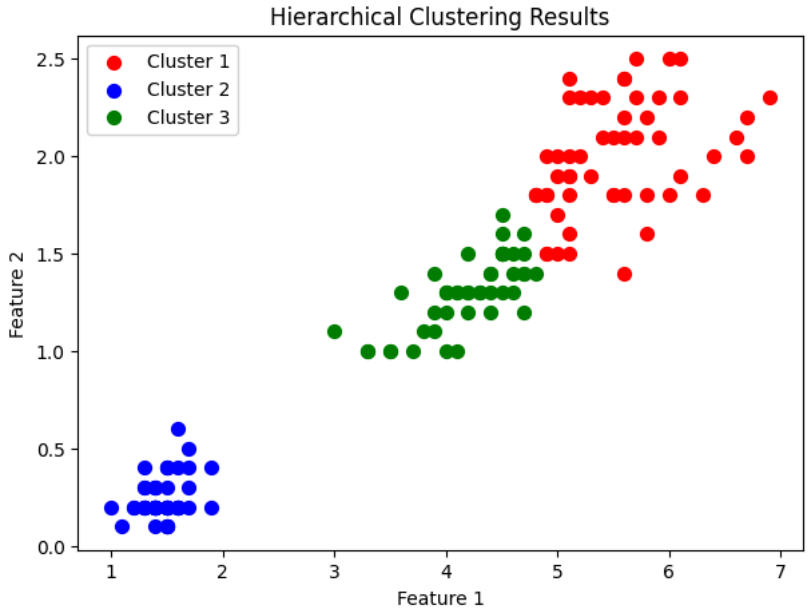

```
Explained Variance Ratio (4 components): [0.44272026 0.18971182 0.09393163 0.06602135]
Total Cumulative Variance: 79.24%
```

```
import pandas as pd
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Load data
iris = pd.read_csv('/content/iris dataset.csv')

# Use Petal Length & Petal Width
X = iris[['petal_length', 'petal_width']].values

# K-Means
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
y_kmeans = kmeans.fit_predict(X)

# Plot clusters
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')

# Centroids
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1],
            c='red', s=200, alpha=0.75,
            marker='X', label='Centroids')

plt.title("K-Means Clustering (Iris Petal Data)")
plt.xlabel("Petal Length (cm)")
plt.ylabel("Petal Width (cm)")
plt.legend()
plt.show()
```

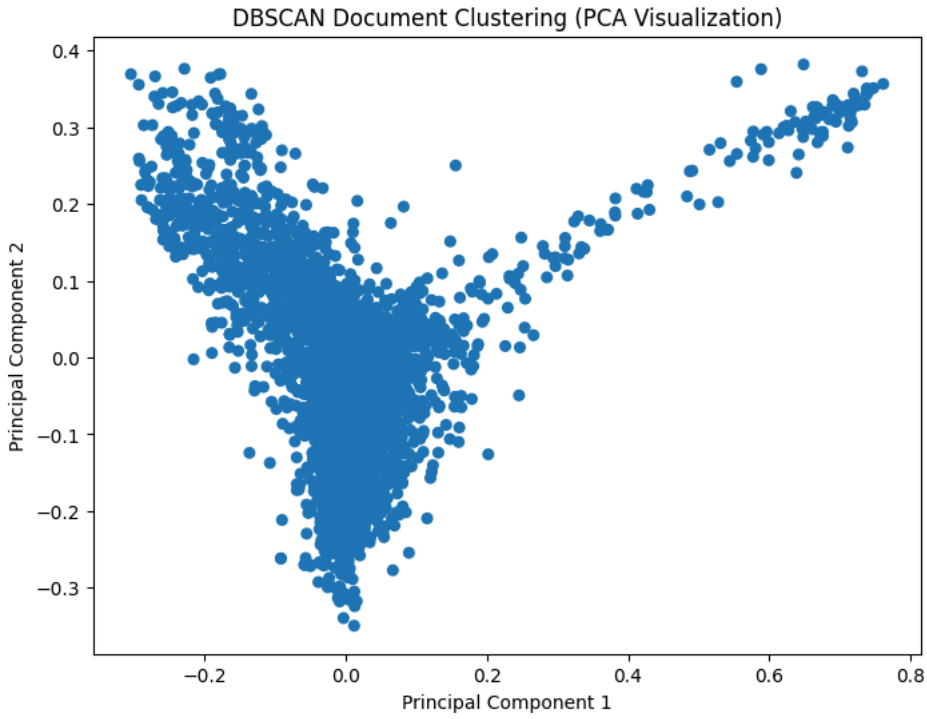



```
import matplotlib.pyplot as plt
import scipy.cluster.hierarchy as sch
from sklearn.cluster import AgglomerativeClustering

# 1. Dendrogram
plt.figure(figsize=(7, 5))
plt.title("Dendrogram (Ward's Method)")
sch.dendrogram(sch.linkage(X, method='ward'))
plt.xlabel('Sample Index')
plt.ylabel('Euclidean Distance')
plt.show()

# 2. Agglomerative Clustering
hc = AgglomerativeClustering(n_clusters=3, metric='euclidean', linkage='ward')
y_hc = hc.fit_predict(X)

# Plot clusters
plt.figure(figsize=(7, 5))
plt.scatter(X[y_hc == 0, 0], X[y_hc == 0, 1], s=50, c='red', label='Cluster 1')
plt.scatter(X[y_hc == 1, 0], X[y_hc == 1, 1], s=50, c='blue', label='Cluster 2')
plt.scatter(X[y_hc == 2, 0], X[y_hc == 2, 1], s=50, c='green', label='Cluster 3')

plt.title('Hierarchical Clustering Results')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.show()
```



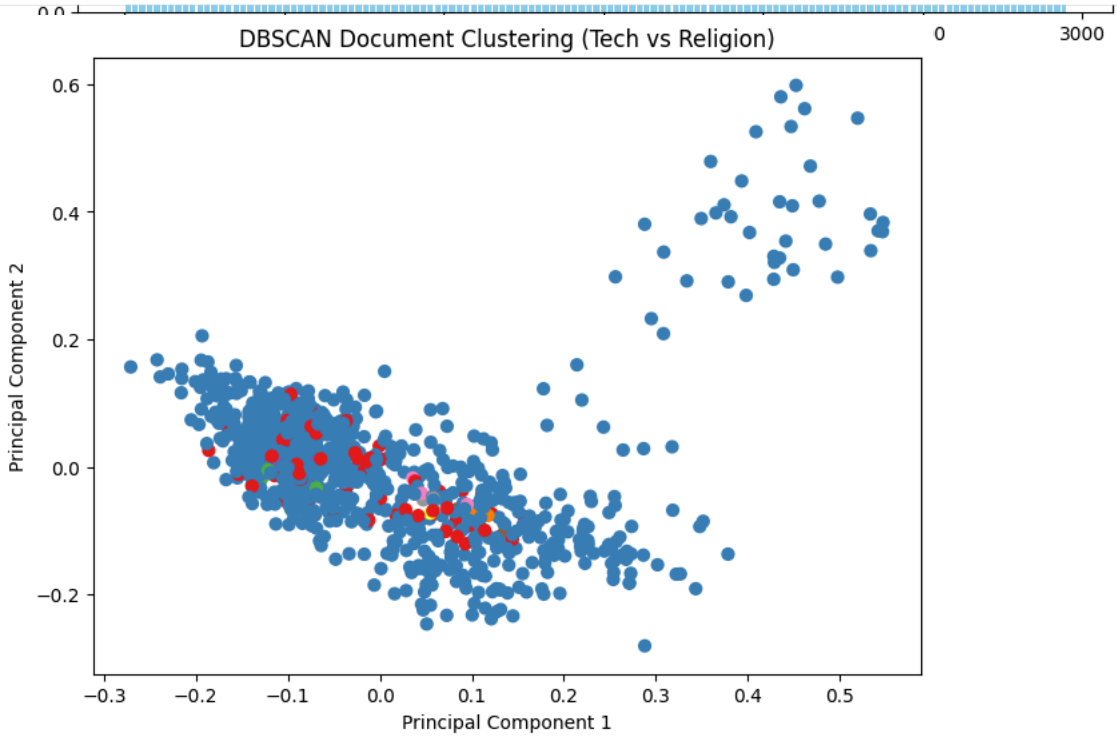



```
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import DBSCAN
from sklearn.datasets import fetch_20newsgroups
from sklearn.decomposition import PCA

# Load text data
categories = ['sci.space', 'comp.graphics', 'sci.med']
newsgroups = fetch_20newsgroups(subset='all', categories=categories)

# Convert text to TF-IDF vectors
vectorizer = TfidfVectorizer(stop_words='english', max_features=500)
X_tfidf = vectorizer.fit_transform(newsgroups.data).toarray()

# Apply DBSCAN
dbscan = DBSCAN(eps=0.8, min_samples=5, metric='cosine')
clusters = dbscan.fit_predict(X_tfidf)

# Reduce to 2D using PCA for visualization
pca_docs = PCA(n_components=2).fit_transform(X_tfidf)

# Plot
plt.figure(figsize=(8, 6))
plt.scatter(pca_docs[:, 0], pca_docs[:, 1], c=clusters, cmap='tab10', s=30)
plt.title("DBSCAN Document Clustering (PCA Visualization)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()
```




```
from sklearn.cluster import OPTICS
import numpy as np

# Implementation of OPTICS
optics_model = OPTICS(min_samples=10, xi=0.05, metric='cosine')
optics_model.fit(X_tfidf)

# Visual: Reachability Plot
space = np.arange(len(X_tfidf))
reachability = optics_model. reachability_[optics_model.ordering_]
labels = optics_model. labels_[optics_model.ordering_]

plt.figure(figsize=(10, 7))
plt.bar(space, reachability, color='skyblue' )
plt. title('OPTICS Reachability Plot (Document Density) ')
plt.ylabel ('Reachability Distance' )
plt.xlabel ('Points ordered by density' )
plt.show()
```

```
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import DBSCAN
from sklearn.datasets import fetch_20newsgroups
from sklearn.decomposition import PCA

# Load dataset
categories = ['comp.graphics', 'talk.religion.misc']
data = fetch_20newsgroups(subset='train', categories=categories)

# Convert text into TF-IDF vectors
vectorizer = TfidfVectorizer(stop_words='english', max_features=1000)
X_text = vectorizer.fit_transform(data.data).toarray()

# DBSCAN clustering
dbscan = DBSCAN(eps=0.7, min_samples=3, metric='cosine')
labels = dbscan.fit_predict(X_text)

# PCA for 2D visualization
pca_text = PCA(n_components=2).fit_transform(X_text)

# Plot
plt.figure(figsize=(8, 6))
plt.scatter(pca_text[:, 0], pca_text[:, 1], c=labels, cmap='Set1', s=40)
plt.title('DBSCAN Document Clustering (Tech vs Religion)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```




```
from sklearn.cluster import OPTICS
```